

Comprendre le code de

IA LOCAL

Cours d'initiation : Python & SQL expliqués avec ton propre code

Conçu pour un débutant complet.

Chaque notion est expliquée puis illustrée par un vrai extrait de tes fichiers
database.py, *main.py* et *voir_base.py*.

Projet : IA LOCAL — Synchronisation PC ↔ Android

Sommaire

Chapitre 1 — Les bases de Python à travers ton code

- 1.1 Un fichier .py, c'est quoi
- 1.2 Importer du code (import / from ... import)
- 1.3 Les variables et les types de données
- 1.4 Écrire une fonction (def, paramètres, return)
- 1.5 Les valeurs par défaut
- 1.6 Les f-strings (afficher du texte avec des variables dedans)
- 1.7 Les conditions : if / elif / else
- 1.8 Les boucles : for
- 1.9 Gérer les erreurs : try / except
- 1.10 Le bloc with (ouvrir/fermer un fichier proprement)
- 1.11 pathlib : manipuler des fichiers et des dossiers
- 1.12 Listes, dictionnaires, tuples, sets
- 1.13 Les docstrings et les commentaires

Chapitre 2 — Les bases de SQL à travers ta base de données

- 2.1 Une base de données, une table, une ligne, une colonne
- 2.2 CREATE TABLE : construire une table
- 2.3 Les types de colonnes utilisés dans samyai.db
- 2.4 Les contraintes : PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT
- 2.5 SELECT : lire des données
- 2.6 WHERE : filtrer les résultats
- 2.7 INSERT INTO : ajouter une ligne
- 2.8 UPDATE : modifier une ligne existante
- 2.9 ON CONFLICT ... DO UPDATE : l'upsert
- 2.10 Le point (?) : se protéger de l'injection SQL

Chapitre 3 — Lecture guidée de tes 3 fichiers

- 3.1 database.py, ligne par ligne
- 3.2 main.py : calculer_hash()
- 3.3 main.py : comparer_etat()
- 3.4 main.py : scanner_et_synchroniser_bdd()
- 3.5 voir_base.py

Chapitre 4 — Glossaire rapide

Chapitre 1 Les bases de Python à travers ton code

Python est un langage de programmation : un ensemble de règles qui permet d'écrire des instructions que l'ordinateur va exécuter dans l'ordre, du haut vers le bas. Un fichier qui se termine par **.py** (comme **main.py**) est un fichier Python : l'ordinateur le lit et exécute chaque ligne les unes après les autres.

À retenir

Dans ce cours, chaque notion est suivie d'un vrai morceau de ton code. Tu n'as pas besoin de tout retenir d'un coup : reviens ici dès qu'une ligne de ton projet te paraît obscure.

1.1 — Un fichier .py, c'est quoi

Ton projet contient 3 fichiers Python : **database.py** (crée la base de données), **main.py** (scanne les fichiers et les compare à la base) et **voir_base.py** (affiche le contenu de la base). Chacun peut définir des fonctions réutilisables, et éventuellement du code qui s'exécute automatiquement quand on lance le fichier.

1.2 — Importer du code (import / from ... import)

Python fournit des « boîtes à outils » toutes prêtes qu'on appelle des **modules**. Pour utiliser les outils d'un module, il faut d'abord l'**importer** tout en haut du fichier.

PYTHON

```
from pathlib import Path
from datetime import datetime
import hashlib
import sqlite3

from database import creer_base, NOM_BASE
```

Extrait de *main.py*, lignes 1-6

- **import sqlite3** — importe le module qui sait parler à une base de données SQLite.
- **import hashlib** — importe le module qui sait calculer des empreintes (hash) de fichiers.
- **from pathlib import Path** — importe uniquement l'outil **Path** depuis le module **pathlib** (au lieu de tout le module).
- **from database import creer_base, NOM_BASE** — importe deux éléments définis dans *ton propre fichier database.py*. En Python, un fichier peut en importer un autre du moment qu'ils sont dans le même dossier.

1.3 — Les variables et les types de données

Une variable est un nom qui pointe vers une valeur. Les types que tu croises le plus dans ton code :

- **str** (texte) — "samyai.db", "PC"
- **int** (nombre entier) — une taille de fichier en octets, par exemple 85554
- **bool** (vrai/faux) — le résultat d'un test comme `fichier.is_file()`

- **None** — « rien », utilisé quand une valeur n'existe pas encore
- **tuple** — un petit paquet de valeurs liées, par exemple (taille, hash, statut)

PYTHON

```
NOM_BASE = "samyai.db"
```

Extrait de database.py, ligne 3

1.4 — Écrire une fonction (def, paramètres, return)

Une fonction est un petit programme réutilisable, à qui on donne des **paramètres** en entrée et qui peut **renvoyer** (return) un résultat. On la définit avec le mot-clé **def**.

PYTHON

```
def calculer_hash(fichier):
    h = hashlib.sha256()
    try:
        with open(fichier, "rb") as f:
            while morceau := f.read(4096):
                h.update(morceau)
    return h.hexdigest()
except FileNotFoundError:
    return None
```

Extrait de main.py, lignes 9-18

Ici, **calculer_hash** est le nom de la fonction, **fichier** est son unique paramètre. Elle renvoie soit une empreinte SHA-256 (une chaîne de texte), soit **None** si le fichier n'existe pas. On l'appelle ensuite simplement en écrivant `calculer_hash(mon_fichier)`.

1.5 — Les valeurs par défaut

Un paramètre de fonction peut avoir une valeur par défaut : si l'appelant ne la précise pas, Python utilise celle-ci automatiquement.

PYTHON

```
def scanner_et_synchroniser_bdd(dossier, chemin_base=NOM_BASE, appareil="PC"):
    ...
```

Extrait de main.py, ligne 47

Si tu appelles `scanner_et_synchroniser_bdd("./test")`, Python utilisera automatiquement `chemin_base="samyai.db"` et `appareil="PC"`, sans que tu aies à les répéter à chaque appel.

1.6 — Les f-strings

Un **f-string** (chaîne de texte préfixée par **f**) permet d'insérer la valeur d'une variable directement dans un texte, entre accolades `{ }`.

PYTHON

```
print(f"{len(changements)} changement(s) synchronise(s) dans la BDD locale.")
print(f" - [{c['action'].upper()}] {c['chemin']}")
```

Extrait de main.py, lignes 128 et 130

`len(changements)` compte le nombre d'éléments dans la liste `changements`, et sa valeur est directement injectée dans le texte affiché.

1.7 — Les conditions : `if / elif / else`

Une condition exécute un bloc de code seulement si une expression est vraie.

PYTHON

```
if ancien is None:
    return "nouveau"

if ancien_statut == "supprime":
    return "nouveau"

if ancienne_taille == taille_actuelle and ancien_hash == hash_actuel:
    return "inchange"

return "modifie"
```

Extrait de `main.py`, fonction `comparer_etat` (lignes 31-44)

Toute la logique de détection de ton projet repose sur cet enchaînement de tests : chaque **if** est vérifié dans l'ordre, et dès que l'un est vrai, la fonction s'arrête et renvoie sa valeur.

1.8 — Les boucles : `for`

Une boucle **for** répète un bloc de code pour chaque élément d'une collection (une liste, les fichiers d'un dossier, les lignes d'un résultat SQL...).

PYTHON

```
for fichier in racine.rglob("*"):
    if not fichier.is_file():
        continue
    ...
```

Extrait de `main.py`, lignes 58-60

Ici, `racine.rglob("*")` renvoie tous les fichiers et dossiers contenus (même dans les sous-dossiers), et la boucle traite chacun un par un. **continue** veut dire « passe directement à l'élément suivant ».

1.9 — Gérer les erreurs : `try / except`

Un bloc **try / except** permet de tenter une action risquée (ex : ouvrir un fichier) sans que le programme plante si ça échoue.

PYTHON

```
try:
    with open(fichier, "rb") as f:
        ...
    return h.hexdigest()
except FileNotFoundError:
    return None
```

Extrait de `main.py`, fonction `calculer_hash`

Si le fichier a disparu entre le moment où on l'a repéré et le moment où on l'ouvre, Python lève une erreur **FileNotFoundError**. Plutôt que de faire planter tout le programme, le code « attrape » cette erreur et renvoie **None** proprement.

1.10 — Le bloc `with` (ouvrir/fermer un fichier proprement)

with ouvre une ressource (ici, un fichier) et garantit qu'elle sera bien refermée à la fin du bloc, même en cas d'erreur — sans que tu aies à écrire `f.close()` toi-même.

PYTHON

```
with open(fichier, "rb") as f:
    while morceau := f.read(4096):
        h.update(morceau)
```

Extrait de main.py, lignes 13-15

"rb" veut dire « read binary » : on lit le fichier en mode binaire brut (utile pour tout type de fichier, pas seulement du texte). La ligne `while morceau := f.read(4096)` lit le fichier par petits blocs de 4096 octets à la fois, pour ne pas charger un fichier énorme entièrement en mémoire d'un coup.

1.11 — `pathlib` : manipuler des fichiers et des dossiers

Le module **pathlib** (via la classe **Path**) représente un chemin de fichier comme un objet, avec des outils intégrés plutôt que de jongler avec du texte brut.

PYTHON

```
racine = Path(dossier)
...
chemin_relatif = str(fichier.relative_to(racine))
stats = fichier.stat()
taille = stats.st_size
extension = fichier.suffix.lstrip(".").lower()
```

Extrait de main.py, lignes 54, 62, 65-68

- **fichier.relative_to(racine)** — transforme un chemin complet en chemin relatif au dossier scanné (ex : `test\\unnamed.png`).
- **fichier.stat()** — renvoie les informations système du fichier (taille, date de modification...).
- **fichier.suffix** — renvoie l'extension avec le point, ex `".pdf"` — **.lstrip(".")** retire ce point, **.lower()** met en minuscules.

1.12 — Listes, dictionnaires, tuples, sets

Quatre façons de regrouper plusieurs valeurs, toutes utilisées dans ton code :

- **Liste** `[]` — une collection ordonnée, qu'on peut parcourir et à laquelle on peut ajouter des éléments : `changements_detectes = []`.
- **Dictionnaire** `{}` — des paires clé/valeur : `{"action": statut, "chemin": chemin_relatif}` permet d'accéder ensuite à `c["action"]`.
- **Tuple** `()` — comme une liste, mais qu'on ne modifie pas : `(taille, hash, statut)`, renvoyé par une requête SQL.

- **Set** `set()` — une collection sans doublons ni ordre, idéale pour tester rapidement « est-ce que cet élément est dedans » : `fichiers_actuels = set()`.

PYTHON

```
ancienne_taille, ancien_hash, ancien_statut = ancien
```

Extrait de main.py, ligne 34 — « décomposer » un tuple en 3 variables d'un coup

1.13 — Les docstrings et les commentaires

Un texte entre triples guillemets juste après **def** est une **docstring** : elle documente ce que fait la fonction. Une ligne commençant par **#** est un commentaire, ignoré par Python, qui sert uniquement à expliquer une décision au lecteur humain.

PYTHON

```
def comparer_etat(ancien, taille_actuelle, hash_actuel):  
    """Détermine l'état d'un fichier par rapport à ce qui est enregistré en base."""  
    ...  
    # Un fichier qui réapparaît après avoir été marqué supprimé doit être re-synchronisé  
    if ancien_statut == "supprime":  
        return "nouveau"
```

Extrait de main.py

Chapitre 2 Les bases de SQL à travers ta base de données

SQL (Structured Query Language) est le langage utilisé pour parler à une base de données — ici, ta base **samyai.db**, gérée par **SQLite** (un moteur de base de données léger qui stocke tout dans un seul fichier, sans serveur à installer).

2.1 — Une base de données, une table, une ligne, une colonne

Imagine un tableau Excel. La **base de données** est le classeur entier, une **table** est une feuille (ici, ta table s'appelle **fichiers**), une **colonne** est un en-tête de colonne (nom, chemin, taille...) et une **ligne** représente un fichier suivi.

2.2 — CREATE TABLE : construire une table

L'instruction **CREATE TABLE** définit la structure (le « schéma ») d'une table : son nom et la liste de ses colonnes avec leur type.

SQL

```
CREATE TABLE IF NOT EXISTS fichiers (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    nom TEXT NOT NULL,  
    chemin TEXT NOT NULL UNIQUE,  
    extension TEXT,  
    taille INTEGER NOT NULL,  
    date_modification TIMESTAMP,  
    hash TEXT,  
    appareil TEXT NOT NULL DEFAULT 'PC',  
    statut TEXT NOT NULL DEFAULT 'nouveau',  
    mis_a_jour_le TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    derniere_sync_le TIMESTAMP  
)
```

Extrait de database.py, lignes 12-24

IF NOT EXISTS évite une erreur si la table existe déjà : la commande ne fait rien dans ce cas, au lieu de planter.

2.3 — Les types de colonnes utilisés dans samyai.db

- **INTEGER** — un nombre entier (ex : taille, la taille du fichier en octets).
- **TEXT** — du texte (ex : chemin, hash, statut).
- **TIMESTAMP** — une date/heure, stockée comme du texte par SQLite (ex : mis_a_jour_le).

2.4 — Les contraintes : PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT

- **PRIMARY KEY AUTOINCREMENT** — identifiant unique de chaque ligne, généré automatiquement (1, 2, 3...). C'est la carte d'identité de la ligne.
- **NOT NULL** — cette colonne doit toujours avoir une valeur, elle ne peut pas rester vide.
- **UNIQUE** — deux lignes ne peuvent pas avoir la même valeur dans cette colonne. Sur chemin, ça empêche d'avoir deux fois le même fichier en base — c'est ce qui manquait dans la version initiale et

créait des doublons.

- **DEFAULT** — valeur utilisée automatiquement si on n'en précise pas une, ex : statut vaut 'nouveau' par défaut.

Pourquoi c'est important

Sans **UNIQUE** sur chemin, rien n'empêche SQLite d'enregistrer 3 fois le même fichier. C'est exactement le bug qu'on a corrigé en Phase 2 de ton projet.

2.5 — SELECT : lire des données

SELECT récupère des données. **SELECT *** veut dire « toutes les colonnes », ou on peut lister précisément celles qu'on veut.

SQL

```
SELECT * FROM fichiers
```

Extrait de `voir_base.py`, ligne 7

SQL

```
SELECT taille, hash, statut FROM fichiers WHERE chemin = ?
```

Extrait de `main.py`, ligne 71

2.6 — WHERE : filtrer les résultats

WHERE ajoute une condition pour ne récupérer (ou modifier) que certaines lignes, au lieu de toute la table.

SQL

```
SELECT chemin FROM fichiers WHERE statut != 'supprime'
```

Extrait de `main.py`, ligne 105 — toutes les lignes SAUF celles déjà marquées supprimées

2.7 — INSERT INTO : ajouter une ligne

INSERT INTO ajoute une nouvelle ligne dans une table, en précisant les colonnes puis les valeurs correspondantes dans le même ordre.

SQL

```
INSERT INTO fichiers (  
    nom, chemin, extension, taille,  
    date_modification, hash, appareil,  
    statut, mis_a_jour_le  
)  
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, CURRENT_TIMESTAMP)
```

Extrait de `main.py`, lignes 88-89 (colonnes réparties sur plusieurs lignes ici pour la mise en page)

2.8 — UPDATE : modifier une ligne existante

UPDATE change la valeur d'une ou plusieurs colonnes sur les lignes qui correspondent à la condition **WHERE**. Sans **WHERE**, la modification s'appliquerait à toute la table !

SQL

```
UPDATE fichiers
SET statut = 'supprime', mis_a_jour_le = CURRENT_TIMESTAMP
WHERE chemin = ?
```

Extrait de main.py, lignes 111-112

2.9 — ON CONFLICT ... DO UPDATE : l'upsert

« Upsert » = update + insert. Cette clause dit à SQLite : « essaie d'insérer cette ligne, mais si une ligne existe déjà avec le même chemin (grâce à la contrainte **UNIQUE**), mets-la à jour au lieu de planter ».

SQL

```
ON CONFLICT(chemin) DO UPDATE SET
    nom = excluded.nom,
    taille = excluded.taille,
    hash = excluded.hash,
    statut = excluded.statut,
    mis_a_jour_le = CURRENT_TIMESTAMP
```

Extrait de main.py, lignes 90-98 (simplifié)

`excluded.nom` désigne la valeur qu'on essayait d'insérer. Cette clause ne peut fonctionner que grâce à la contrainte **UNIQUE(chemin)** du chapitre 2.4 — sans elle, SQLite refuse la commande.

2.10 — Le point d'interrogation (?) : se protéger de l'injection SQL

Plutôt que de coller directement une valeur dans le texte de la requête, le code utilise des ? comme « trous » à remplir, et fournit les vraies valeurs séparément, dans un tuple.

PYTHON

```
curseur.execute(
    "SELECT taille, hash, statut FROM fichiers WHERE chemin = ?",
    (chemin_relatif,),
)
```

Extrait de main.py, lignes 70-73

Pourquoi c'est important

Si on collait directement le nom de fichier dans le texte SQL (avec un f-string par exemple), un nom de fichier malveillant pourrait casser ou détourner la requête (on appelle ça une **injection SQL**). Les ? empêchent totalement ce problème : c'est la bonne pratique à toujours garder.

Chapitre 3 Lecture guidée de tes 3 fichiers

Maintenant qu'on a vu les briques une par une, relisons chaque fichier dans son ensemble, en reliant chaque bloc aux notions du chapitre 1 et 2.

3.1 — database.py, ligne par ligne

PYTHON

```
import sqlite3

NOM_BASE = "samyai.db"

def creer_base(chemin_base=NOM_BASE):
    """Crée la table fichiers si elle n'existe pas déjà."""
    connexion = sqlite3.connect(chemin_base)
    curseur = connexion.cursor()

    curseur.execute("""
    CREATE TABLE IF NOT EXISTS fichiers ( ... )
    """)

    connexion.commit()
    connexion.close()

if __name__ == "__main__":
    creer_base()
    print("Base de données initialisée.")
```

database.py (schéma SQL raccourci, voir 2.2)

- **NOM_BASE** est une constante (une variable qu'on ne réassigne jamais) qui centralise le nom du fichier de base, réutilisée dans **main.py** via l'import du chapitre 1.2.
- **sqlite3.connect(...)** ouvre (ou crée si absent) le fichier `samyai.db`.
- **connexion.cursor()** crée un « curseur », l'objet qui exécute réellement les commandes SQL.
- **connexion.commit()** valide définitivement les changements — sans lui, rien n'est vraiment enregistré sur le disque.
- **if __name__ == "__main__":** ce bloc ne s'exécute que si on lance `python database.py` directement, pas quand un autre fichier l'importe. C'est ce qui rend **creer_base** réutilisable proprement depuis **main.py**.

3.2 — main.py : calculer_hash()

Vue au chapitre 1.4, 1.9 et 1.10. Elle lit un fichier morceau par morceau et calcule son empreinte SHA-256 — une sorte de « code-barres » unique du contenu du fichier : si un seul octet change, le hash change complètement. C'est ce qui permet de détecter une modification de façon fiable, sans se fier juste au nom ou à la date.

3.3 — main.py : comparer_etat()

Vue au chapitre 1.7. Elle reçoit l'ancien état d'un fichier (ou **None**) et compare avec l'état actuel pour décider s'il est "nouveau", "modifié" ou "inchangé". Elle ne touche jamais à la base de données

elle-même : c'est une fonction de pure décision, facile à tester isolément.

3.4 — main.py : scanner_et_synchroniser_bdd()

La fonction principale, qui orchestre tout :

- 1. Elle s'assure que la table existe (`creer_base(chemin_base)`).
- 2. Elle parcourt tous les fichiers du dossier (boucle **for**, chapitre 1.8).
- 3. Pour chacun, elle interroge la base (**SELECT**, chapitre 2.5) pour connaître son ancien état.
- 4. Elle appelle **comparer_etat()** pour décider du statut.
- 5. Si le statut a changé, elle enregistre la nouvelle ligne (**INSERT ... ON CONFLICT**, chapitre 2.9).
- 6. À la fin, elle repère les fichiers qui ont disparu du disque et les marque "supprime" (**UPDATE**, chapitre 2.8) sans effacer leur ligne, pour garder un historique.

3.5 — voir_base.py

PYTHON

```
import sqlite3

connexion = sqlite3.connect("samyai.db")
curseur = connexion.cursor()
curseur.execute("SELECT * FROM fichiers")
resultat = curseur.fetchall()

for fichier in resultat:
    print(fichier)

connexion.close()
```

voir_base.py (fichier complet)

Le plus simple des trois fichiers : il ouvre la base, exécute un **SELECT *** (chapitre 2.5), récupère tous les résultats avec **fetchall()** (une liste de tuples), puis les affiche un par un avec une boucle **for**. C'est ton outil de débogage rapide pour « voir ce qu'il y a vraiment dans la base ».

Chapitre 4 Glossaire rapide

Fonction — Un bloc de code réutilisable, défini avec **def**, qui peut recevoir des paramètres et renvoyer un résultat avec **return**.

Module — Un fichier Python (ou une bibliothèque) qu'on peut importer avec **import**.

Base de données — Un fichier structuré qui stocke des données organisées en tables, interrogeable avec le langage SQL.

Table — L'équivalent d'une feuille de calcul dans une base de données : des colonnes et des lignes.

Curseur — L'objet Python qui exécute les commandes SQL sur une connexion à la base.

SHA-256 — Un algorithme qui transforme le contenu d'un fichier en une empreinte de longueur fixe : la moindre modification du fichier change complètement l'empreinte.

Hash — Le résultat produit par un algorithme comme SHA-256 : une empreinte numérique.

Schéma — La structure d'une table : la liste de ses colonnes et de leurs types/contraintes.

Contrainte — Une règle imposée à une colonne (NOT NULL, UNIQUE, DEFAULT...).

Upsert — Contraction de « update » et « insert » : insérer une ligne, ou la mettre à jour si elle existe déjà.

Injection SQL — Une attaque où du texte non filtré est interprété comme du code SQL. Évitée en utilisant des paramètres ? plutôt que de coller du texte directement dans la requête.

Docstring — Un texte entre triples guillemets juste après une fonction, qui documente ce qu'elle fait.

Chemin relatif — Un chemin de fichier exprimé par rapport à un dossier de référence, plutôt que depuis la racine du disque.

Et maintenant ?

Chaque fois qu'on avancera dans les prochaines phases du projet (surveillance automatique, communication réseau, transferts...), on pourra étendre ce même cours avec les nouvelles notions rencontrées.